

Antigravity Vibe-Coding Execution Prompts

Stage-by-stage prompts for building safely without letting the coding agent drift from the assessment

How to use this document

Give Antigravity one stage at a time. Require it to inspect existing files before editing, make small reversible changes, run the relevant checks, and stop with a concise report. Do not ask it to build the entire project in one prompt.

Stage 0 — Repository guardrails

Act as a senior frontend engineer. Read the assessment PDF and all project files before changing anything. This is a FRONTEND-ONLY four-step signup-wizard replication. Do not add a backend, database, real OTP provider, auth service, API secrets, or unnecessary dependencies. Create a short implementation checklist that maps every assessment requirement to a screen/component/test. Identify unknown details that must be copied from the reference app instead of inventing them. Do not code yet.

Stage 1 — Scaffold and architecture

Implement only the project foundation using React + TypeScript + Vite. Add routing for Landing, Terms, Signup and Success. Add Tailwind/CSS variables, Poppins, a clean src/app + pages + features/onboarding + components/ui + styles structure, strict TypeScript, ESLint and formatting. Keep the UI minimal for now. Run install, typecheck and production build. Fix all errors before stopping. Report files changed and commands run.

Stage 2 — Design system and responsive shell

Using my captured reference screenshots as the visual source of truth, build the responsive page shell and reusable Button, Field, Select, Progress, Toast/Banner and LoadingSpinner components. Match typography, spacing, colors, radii, shadows and component behavior closely. Mobile first, then tablet and desktop. Do not add trendy effects that are absent from the reference. Add visible keyboard focus and reduced-motion support. Run typecheck/build and stop.

Stage 3 — Landing and Terms

Implement the landing-page mechanism and similar Terms & Conditions page required by the assessment. Reproduce the reference hierarchy and navigation. Optimize background/logo assets, reserve image dimensions to avoid layout shift, use semantic headings, and keep legal text readable on desktop. No backend calls. Verify mobile 320px, tablet 768px and desktop 1440px. Run checks and stop.

Stage 4 — Four-step wizard + validation

Implement the exact four signup steps from my reference notes/screenshots. Use React Hook Form + Zod and a typed wizard store/reducer. Preserve valid data when navigating Back. Add required/format/length/numeric/whitespace validation and cross-field dependencies exactly where relevant. Handle under-18 age explicitly if age is collected. Do not guess fields not shown in the reference. Focus the first invalid field on submit. Run unit tests for schemas plus typecheck/build.

Stage 5 — Verification, loading, failures and success

Add deterministic FRONTEND-ONLY mock async services for email/OTP/profile submission. Show button spinners, prevent duplicate requests, handle stale requests safely, and implement inline plus global errors. Make OTP UX polished: paste support, clear invalid/expired state, resend waiting state and accessible focus behavior, while remaining visually consistent with the reference. Add a documented demo failure trigger and a deterministic success path. Never send personal data to a real service.

Stage 6 — Security, accessibility and optimization

Audit the project as a senior frontend reviewer. Remove console logs, unused packages and dead code. Check that no secrets or real credentials exist. Avoid localStorage for sensitive profile data; use no persistence or minimal sessionStorage only if needed. Test keyboard navigation, aria labels/descriptions, live async feedback, 200% zoom, reduced motion, touch targets, image optimization, font loading, rerenders and layout shift. Fix issues without changing the reference visual design.

Stage 7 — Test and submission hardening

Add/finish Vitest + React Testing Library tests for validation/components and Playwright tests for the complete happy path, invalid email, OTP error, Back navigation, under-age behavior, dependent fields if present, simulated submit failure/retry and success. Run lint, tests, typecheck and production build. Produce a final requirement matrix showing PASS/FAIL and exact evidence. Do not mark anything PASS unless it was actually tested.

Stage 8 — Final manager-impression pass

Perform a final visual QA against the reference screenshots. Prioritize fidelity over redesign. Fix inconsistent spacing, typography, alignment, focus states, mobile keyboard issues, loading-state layout shifts, weak error copy and awkward desktop scaling. Keep motion subtle. Then prepare a concise README section with run commands, architecture, mock behavior, test commands and known assumptions. Do not include secrets or claim a backend exists.

Agent rules to paste at the top of every prompt

- Read before edit. Never overwrite working behavior blindly.
- One stage only; stop after its acceptance checks.
- No backend/database/auth vendor because the assignment explicitly says frontend only.
- No invented signup fields: reference app/screenshots are source of truth.
- No secrets, real OTP, real personal-data transmission or hard-coded credentials.
- Run typecheck/build after every meaningful stage; run relevant tests when introduced.
- Prefer small dependencies and reusable typed components.
- Do not hide errors with ts-ignore, any, disabled lint rules or skipped tests.
- Preserve accessibility and responsive behavior in every edit.
- Summarize exactly what changed, what was tested and any remaining uncertainty.